

Laborator Arhitectura Sistemelor de Calcul - FMI

Matei-Iulian Cocu
matei-iulian.cocu@s.unibuc.ro

Ștefan Chiper
stefan.chiper@s.unibuc.ro

Cuprins

1	Laborator 0x00: Introducere în limbajul de programare Assembly x86	2
1.1	Introducere	2
1.2	Masina virtuala & Git	2
1.3	Limbajul Assembly x86	2
1.4	Registrii	3
2	Laborator 0x01: Probleme avansate în Assembly x86	3
2.1		3
2.2		3
2.3		3
2.4	Lectură suplimentară	3
3	Laborator 0x02: Introducere în RISC-V	3
3.1		3
3.2		3
3.3		3
3.4	Lectură suplimentară	3

1 Laborator 0x00: Introducere în limbajul de programare Assembly x86

1.1 Introducere

Scopul acestui laborator este acela de a va introduce limbajul Assembly x86 si de a va obisnui cu sintaxa AT&T pentru procesoarele x86. In acest sens, accentul este pus, in mare parte, pe insusirea notiunilor elementare de programare intr-un limbaj de asamblare.

1.2 Masina virtuala & Git

1.2.1 Masina virtuala / Windows subsystem for Linux

Acest laborator va fi predat sub Linux, astfel ca daca nu aveti o distributie de Linux ca sistem principal de operare, va trebui sa va instalati cel putin pe o masina virtuala. Recomandarea este sa va luati o imagine de Ubuntu si sa o instalati pe VMWare sau pe VirtualBox. Imaginea sistemului de operare o gasiti pe site-ul de la [Ubuntu](https://ubuntu.com). Doua tutoriale video excelente pentru instalarea mediului de lucru, impreuna cu unele sugestii de setare pentru sistem le gasiti aici:

- Windows & Ubuntu (other Linux-based - Debian) OS: https://youtu.be/XGr6_QV5moU
- MacOS Apple Silicon (Mx Normal/Pro/Max): <https://youtu.be/1kENnRk39JO>

Daca nu ati mai folosit Linux, o investitie buna pentru viitorul vostru este un alt [tutorial](#) putin mai detaliat.

1.2.2 Git

La aceast curs, tot codul sursa scris de voi va fi integrat intr-un sistem pentru controlul versiunilor. Aceasta este o practica standard in industria software. Munca din timpul laboratoarelor va fi salvata pe Git. In acest sens, va recomand sa va faceti un cont pe Github, sa va creati un repository cu numele "Laborator ASC" sau similar, iar toate problemele pe care le lucrati sa le aveti centralizate aici. Cand/daca aveti intrebari legate de programele realizate la acest laborator noi le putem verifica direct online pe Git si va putem sugera solutii.

Un [tutorial](#) video excelent pentru instalarea Git pe sistemul vostru este disponibil online.

1.3 Limbajul Assembly x86

Assembly x86 este un limbaj de programare de nivel scazut (low-level programming language) care permite controlul direct asupra unitatii centrale de calcul (CPU) a unui calculator cu arhitectura de calcul x86. In multe limbaje de programare de nivel inalt compilatorul creeaza prima data cod assembly ca un pas intermediar. Codul sursa assembly este o bucata de text, deci un procesor nu are cum sa interpreteze si sa execute acest text. Assembly este transformat apoi in cod masina (machine code): instructiuni masina binare codate clar (opcode) care pot fi apoi executate de procesor pas cu pas. Vrem sa va atragem atentia ca Assembly x86 are doua sintaxe distincte: Intel (folosit pe sisteme Windows) si AT&T (folosit predominant pe sisteme Unix). In acest laborator folosim doar sintaxa AT&T. Daca intelegeti una dintre sintaxe, cealalta e foarte usor de citit si inteles. Componentele principale ale limbajului Assembly x86 sunt:

- Registrari. Ca in orice limbaj de programare, avem nevoie de variabile cu care sa lucram. Aceste variabile stocheaza date (in cazul nostru, valori pe 32 biti). Putem seta si modifica valorile registrilor si putem realiza operatii cu si intre acesti registri (de exemplu: putem aduna valorile din doi registri si il putem pune intr-un al treilea, sau in unul dintre registrari care deja stocheaza un operand). Unii registri stocheaza o locatie in memorie (de exemplu: trebuie sa stim unde se afla urmatoarea instructiune din program care trebuie executata, deci avem un Instruction Pointer - e defapt un "Instruction Register");
- Instructiuni. Procesorul suporta o serie de operatii intre registrari si locatii din memorie. In primul rand avem nevoie de abilitatea de a muta din/in registrari date in/din memorie. In registrari, putem face operatii matematice, logice, I/O, etc.
- Flaguri. Dupa ce facem o operatie, avem o serie de flag-uri (indicatori) care se activeaza in functie de rezultat (daca rezultatul operatiilor este negativ sau zero, sau daca un overflow s-a produs);
- Adresarea memoriei. In procesor avem doar cativa registri. Daca avem nevoie sa lucram cu un volum mare de date (de exemplu, continutul unui fisier) atunci avem nevoie sa ne putem referi la o adresa din memorie (Random Access Memory - RAM). Accesul la registrari este rapid, dar citirea/scrierea datelor din/in RAM este o operatie mult mai lenta ca timp;
- Stiva programului. Orice program care se afla in executie are o locatie speciala de memorie care se numeste stiva (the program stack). Aceasta este o structura de date de tip Last In First Out (LIFO). Vom vedea mai tarziu exemple unde aceasta structura este folositoare (apelul de functii in Assembly);
- Intreruperi. Programul pe care voi il executati (indiferent de limbajul de programare) nu are acces direct la toate resursele hardware. In cele mai multe cazuri, sistemul de operare controleaza accesul la memorie si periferice. De aceea, avem nevoie sa cerem sistemului de operare sa execute niste operatii pentru programul nostru. Aceasta delegare a muncii catre sistemul de operare se realizeaza prin intreruperi.

1.4 Registrii

Arhitectura procesorului este pe 32 de biti, astfel ca registrii utilizati sunt tot pe 32 de biti. Un registru reprezinta o cantitate mica de spatiu de stocare pe unitatea centrala de procesare, al carui continut poate fi accesat mai rapid decat datele stocate in alt loc (de exemplu, in memorie sau chiar mai lent pe hard-disk).

Registrii arhitecturii x86 sunt:

- cei 8 din imaginea de mai sus, numiti registrii de uz general (General-Purpose Registers);
- un registru pentru flags;
- un registru pointer la instructiunea urmatoare ce va fi executata (Instruction Pointer);
- alti registrii

Observatie: Registrul **BP** poate fi utilizat si ca frame pointer.

Alta observatie: Urmărim tabelul si observam ca registrii pe arhitectura x86 pe 32 de biti au prefixul **E** (extended). In acest caz, registrul **EAX** este format din **AX** (accumulator), **AH** (H - high) si **AL** (L - low). De exemplu, daca in **EAX** stocam valoarea 0, dar punem in **AH** ulterior valoarea 1, atunci intreg registrul **EAX** va stoca valoarea 256 in baza 10.

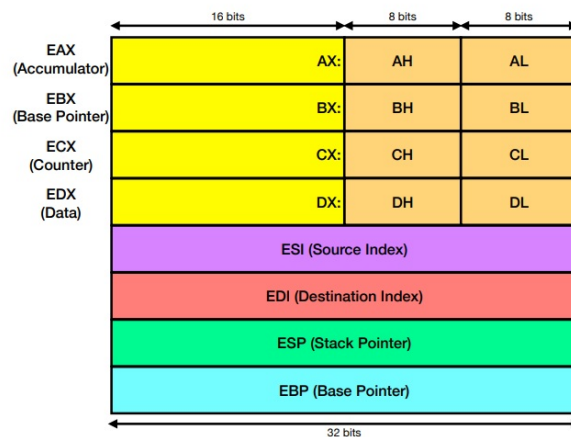


Figure 1: Registrii x86

2 Laborator 0x01: Probleme avansate în Assembly x86

2.1

2.2

2.3

2.4 Lectură suplimentară

3 Laborator 0x02: Introducere în RISC-V

3.1

3.2

3.3

3.4 Lectură suplimentară